

Gain-cell eDRAM 的同状态读写 与刷新服务估算

1 主模式、边界与状态映射

本例选取 GC-04 的 current-programmed 3T1C gain-cell, 采用**二进制端点、伪差分 ACIM**, 结合共同 28 nm 外围能力作参考设计估算。器件证据仍是原 65 nm 工艺, 并非该结构已在 28 nm 复现。写入与计算使用同一电流状态, 周期刷新读取其符号并重新建立相同端点; 不以普通 1T1C DRAM 的访问周期替代 gain-cell, 也不把多级、互补或 stationary latch 认作所有 gain-cell 的必要结构。

共同任务为 128×128 INT8 矩阵乘 INT8 向量; $b_S = b_R = 1$ Byte, 每向量 $B_S = 128$ Byte, resident 逻辑容量 16384 Byte, 输出为 128×24 bit。ACIM 交付经校准的近似部分和及重构结果, 不增加统一算法误差阈值。边界含实现一份矩阵的阵列、输入、局部驱动、ADC、数字重构、输出寄存器以及一个更新控制域; 上下游搬运在外, 不用多个独立计算单元复制吞吐。

一 **bit 两 cell**。原结构每对 3T1C cell 支持差分电流 $-700 \dots 700$ nA。本文只取两个端点: $b = 1$ 存 (700, 0) nA, $b = 0$ 存 (0, 700) nA。八个位平面并行表示 INT8 补码; 每权重 16 cell, 总计 262144 个 3T1C cell、131072 对, 容量只计 16384 Byte。选择八平面、每平面两个 64 行组及两个 64 列组, 共 32 个 64×64 pair 子阵列; 这些是同一矩阵的分片。

保留原生 64 行位线负载, 每次选 16 个输出; 每平面 16 个差分 SAR, 共 128 个, 八次输出分组重新求值, 不跨组保持模拟量。差分 ADC 同时接收两支路, 一对 cell 不需要两次独立转换。128 对写驱动各含选支路、粗写与精写电流路径, 可同时更新 256 cell, 最高目标电流之和为 128×700 nA; 复制参考电流、瞬态粗写供流及选通需要相应驱动资源。128-bit 数据接口本身不能推出这些并行度。

对一个 64 行组, 令输入 bit 为 x_i , 端点符号 $s_i = 2b_i - 1$ 。归一化差分部分和 $z = \sum x_i s_i$ 的满范围是 $-64 \dots 64$, 共 129 级; 二进制部分和为

$$q = \frac{z + \sum_i x_i}{2}. \quad (1)$$

每个输入/行组的 popcount 可广播到输出重构通道。16 条数字通道的两拍预算须容纳该偏移/缩放、八权重位平面合并及输入位权累加; 补码符号位取负权 -2^7 。这与共同约 8 有效 bit 的 129 级部分和目标相容, 仍不意味着模拟结果精确无误。

2 原文锚点与读写预算

证据	原值与条件	在本例中的使用
GC-04 p.6, Fig.8-9	每个 3T1C cell 八级、0-700 nA; 两 cell 构成 15 个有符号等级。10 fF 存储电容; FF、80 °C 的漂移为仿真。	采用 ± 700 nA 两端点; 保留双支路更新。
GC-04 pp.6-7, Fig.10	64 $\times$ 64 pair 宏; 完整 180 ns 含预充、DTC/MAC 与 5-bit SAR。原阵列预充 0.9 V, 数字 1.0 V, ADC 0.7 V。	64 行负载与完整周期量级锚; 不直接当作新 ADC 的实测周期。
GC-04 pp.7-10, Fig.11-12、16	粗写 5 ns; Fig.12 的 60 ns 是模拟。实测扫精写 10-70 ns, Fig.16 显示总 65 ns 时各等级达到目标动态范围, 75 ns 为另一完整测试点。	短/参考完整编程 65 ns, 长 75 ns; 不再叠一次粗写。
GC-04 p.10, Fig.17-18	全部写 +7/ - 7, 调 RWL 脉宽逐行 ADC 读回; 0.4 ms 内 99.7% 测试 cell 漂移小于 1 个原 ADC LSB。	端点保持的条件依据; 不换算为 100 nA, 也不声称全部 bit 无错。

表 1: 关键定位按本地 PDF 页序; 详细证据与其他来源见 `notes/evidence.zh.md`。

读侧是明确的工程适配。原 180 ns 配 5-bit SAR, 不能直接证明共享名义 10-bit、约 8 有效 bit SAR 的速度。本文给专用选通、积分、建立和恢复选择 $t_m = 100/150/200$ ns; 通过共享 JSON 分别加实际 $T_I = 2/5/10$ ns、 $T_A = 10/20/50$ ns, 得到

$$C_A = T_I + t_m + T_A = 112/175/260 \text{ ns.} \quad (2)$$

这些不是把原 180 ns 拆成实测分量, 而是由其完整量级支持的独立新前端预算。公共 ADC 时隙真实传播; 原 180 ns 不再作为第四阶段相加。电路仍须验证 64 行负载、差分 ADC 共模/量程、129 级分辨能力和局部控制。

刷新单行读不能沿用短 MAC 脉冲。按原文调节 RWL 的机制, 本文选择 MAC 单位脉冲 1 ns, 刷新单 pair 端点读脉冲 64 ns; 64 $\times$ 700 nA $\times$ 1 ns 与 700 nA $\times$ 64 ns 均为 44.8 fC, 使单行读使用与满幅部分和相当的积分量。64 ns 已包含在 t_m , 另有 36/86/136 ns 供选通、建立和恢复。每个读出支路的有效积分负载选择 200 fF (含位线与采样适配), 44.8 fC 引起最大 224 mV 摆幅, 差分 ADC 需覆盖 ± 224 mV。128 路双支路合计 51.2 pF; 该负载需在计入寄生、CDAC 接入与隔离缓冲后验证。编程时用局部开关隔离新增积分/采样电容, 将写端含开关寄生的负载保持在原约 50 fF 量级, 才沿用 65 ns 写锚; 重接建立已在专用读槽内。两脉宽与电容均为工程选择, 不是原文测值; 需要相应积分电容/ADC 量程、脉冲精度、非选中泄漏及噪声条件。若不能在此槽内完成, 不得维持本表刷新率。

3 完整求值、直接更新及维护计数

共同 API 的 ACIM 配置只将 128 行改为 64 行，其余采用 R0 的逐 bit 输入、八平面并行、16 ADC/平面、16 数字通道及每轮两拍。于是

$$N_E = N_C = N_{\text{rec}} = 8 \times 2 \times 8 = 128, \quad (3)$$

$$N_{\text{dig}} = 256, \quad N_{\text{scalar}} = 16384, \quad (4)$$

$$C_S = 128C_A + 256T_D + 2T_D. \quad (5)$$

C_S 为无周期维护时完成一次向量的资源占用；捕获和提交共两拍。求值后进行 ADC 和数字重构，再处理下一组，无跨向量流水。参考时 $C_S = 128 \times 175 + 258 \times 5 = 23690$ ns。

resident 一次完成 16 Byte。对一个逻辑行的 16 个输出，同时写八个位平面。每 bit 两个互补支路选择 bit，700 nA 电流幅度本地固定，因此实际装入 $16 \times 16 = 256$ 个控制 bit，需要两个 128-bit 数据拍；首拍随命令装入。准备、完成和后续一拍数据合计 $3T_D$ 。128 个真实 pair 驱动同时完成两支路的电压粗写与电流精写，完整终点为所选电流状态已建立且控制释放：

$$B_R = 16 \text{ Byte}, \quad C_R = 3T_D + P, \quad P = 65/65/75 \text{ ns}. \quad (6)$$

参考 $C_R = 15 + 65 = 80$ ns。原结构采用电流反馈建立目标，不需要另造 NVM 式 pulse/verify 重试；65 ns 的传输函数建立也不是全 cell BER 保证。内部编码翻倍不增加逻辑 payload，粗写 5 ns 已含在 P 。本地顺序更新 1024 个组即可建立整份矩阵，无 native page 或 erase 限制；仅乘 payload 而漏乘装载、控制或物理写步骤均不合法。

刷新完整覆盖本例的真实容量。每一逻辑行按 16 输出分组，八平面的 128 个 pair 同时读出；每个 ADC 配一条本地符号译码逻辑，128 路并行一拍，将 256 个支路控制 bit 装入并重复上述完整写入。总计

$$N_F = 128 \times (128/16) = 1024, \quad (7)$$

$$C_F = C_A + T_D + C_R, \quad (8)$$

$$T_{\text{ref}} = N_F C_F. \quad (9)$$

一轮读 131072 个 pair，重写 262144 个 cell。参考 $C_F = 175 + 5 + 80 = 260$ ns，故 $T_{\text{ref}} = 266240$ ns。刷新内部读出/重写的逻辑 workload payload 为零，不计入 Q_R 。原文 $64 \times 65 \text{ ns} = 4.16 \mu\text{s}$ 只计其原宏逐行重写，未证明本例所需符号读出与有限 ADC 调度已包含，不能据此给本例套 1.1% 折扣。

4 刷新调度、可靠性与配对结果

取周期 $T_F = 400000$ ns, 按固定顺序覆盖全部组。按固定起点和固定顺序刷新; 在每个预定刷新开始前 G 时间停止发新批, 保证同一 cell 的写回间隔不超过 $400 \mu\text{s}$, 而非等当前批结束后再移动刷新起点。streaming 在求值、转换、数字重构完成后暂停, 保存数字累加寄存器, resident 在完整更新组结束后暂停。无跨维护窗口的模拟保持。最大不可中断组长为 $G = C_A + 2T_D$, 三情景为 116/185/280 ns; 每周只扣一次该禁止启动窗口的最大空隙。冷启动需先建立有效状态和固定刷新相位; 本表描述其后的稳态。

$$\alpha = 1 - \frac{T_{\text{ref}} + G}{T_F} > 0, \quad \Delta_S = \frac{C_S}{\alpha}, \quad \Delta_R = \frac{C_R}{\alpha}. \quad (10)$$

这里的 Δ 是可跨周期分批执行的长期平均服务间隔, 不是每请求固定 latency。落在刷新区间的请求会等待, 其完成时间取决于到达相位; 有用服务不足以填满间隙时也不保证达到表内能力。两路分别服务, 刷新均占其共用 ADC、读位线、控制及写驱动。采用固定刷新策略, 不借 workload 更新抵扣维护; 纯连续全矩阵新写可能本身满足保持期限, 任意局部更新则仍要维护其余状态; 能合并刷新与新写入的调度应重算占用, 本文未将潜在抵扣算作收益。

情景	C_A (ns)	P (ns)	C_S (ns)	C_R (ns)	T_{ref} (μs)	可用率
短	112	65	14852	71	189.440	52.611%
参考	175	65	23690	80	266.240	33.394%
长	260	75	35860	105	384.000	3.930%

表 2: 无维护时的占用 C_S, C_R 与每 0.4 ms 全矩阵刷新时间; 可用率再扣批边界余量 116/185/280 ns。

情景	Δ_S (μs)	Δ_R (ns)	ρ (GB/s)	τ (GB/s)	RI*
短	28.2298	134.9528	0.004534	0.118560	0.038244
参考	70.9414	239.5658	0.001804	0.066787	0.027016
长	912.4682	2671.7557	0.000140	0.005989	0.023424

表 3: 包含固定刷新预留后的长期平均服务间隔; $B_S = 128$ Byte、 $B_R = 16$ Byte, GB=10⁹ Byte。

参考 $\alpha = (400000 - 266240 - 185)/400000 = 0.3339375$, 因此

$$\rho = 128\alpha/23690 = 0.00180431 \text{ GB/s}, \quad (11)$$

$$\tau = 16\alpha/80 = 0.0667875 \text{ GB/s}, \quad (12)$$

$$\text{RI}^* = (128/16)(80/23690) = 0.0270156. \quad (13)$$

参考整矩阵顺序更新原始占用 $1024 \times 80 = 81920$ ns; 采用相同维护预留后的平均占用为 $81920/\alpha$ 。相同聚合规则给出同一个 τ , 不是以整矩阵字节替换 B_R 却保留 80 ns。

5 结果解释与一个资源对照

主情景的 ρ 约为 1.40×10^{-4} – 4.54×10^{-3} GB/s, τ 约为 0.00598–0.119 GB/s, 成对 ridge 为 0.0234–0.0383。读侧主要受 128 次模拟求值/转换顺序占用限制, 写侧主要受完整电流建立限制; 整矩阵刷新进一步消耗 47.4%–96.0% 的周期。长情景距离 $T_{\text{ref}} = T_F$ 不远, 未经验证的单行读延长可能使该组织无法满足所选周期。

同一情景内刷新不改变 ridge, 源于所选固定刷新策略对两路乘同一可用率; 三个不同读写预算的 ridge 仍有约 1.6 倍变化。刷新在 ρ/τ 中抵消, 却大幅降低两种实际能力; 不能将这一抵消称为介质对外围或保持条件普遍稳健, 也不能忽略共享资源争用而声称两路峰值可同时实现。

将输出组从 16 增至 32。仅在参考外围作一个明确对照: 差分 ADC 从 128 增至 256, pair 写驱动同步从 128 增至 256; 保持相同 64 行负载和 175 ns 读槽, 本地符号译码也随 ADC 扩至 256 路, 仍一拍; 16 条数字输出通道只用于 MVM 重构。求值/转换批数降至 64, 数字重构仍 256 拍, $C_S = 12490$ ns。每事务写 32 Byte、512 个支路控制 bit, 四个数据拍, $C_R = 5T_D + 65 = 90$ ns。全矩阵刷新需 512 组, 每组 $175 + 5 + 90 = 270$ ns, 另扣一组 195 ns 的边界空隙。

配置	ADC/写对驱动	N_E	刷新 (μs)	ρ (GB/s)	τ (GB/s)	RI*
16 输出	128/128	128	266.24	0.001804	0.066787	0.027016
32 输出	256/256	64	138.24	0.006701	0.232502	0.028823

表 4: 仅参考时隙的资源对照; 32 输出配置每事务完成 32 Byte, 数据接口仍为 128 bit、数字通道仍为 16。

该资源对照的 ρ 约为主参考的 3.7 倍, 而非简单两倍: 求值批数减少, 又降低了真实刷新占用。 τ 也改变, 因为载入拍数、有效写 payload、驱动宽度和维护成本同时变化。它不是主模式默认成绩, 更不是独立阵列复制收益; 所有 cell 仍表示同一份矩阵。

保持与刷新正确性的边界。选择二进制端点使判决阈值为零, 理想幅度距阈值 700 nA, 即七个 100 nA 编程电流间隔; 这与原文的 ADC LSB 是不同单位。p.10 的 99.7%/<1 LSB 统计支持端点保持, 但没有证明余下尾部、读噪声或反复刷新永不跨零。数字符号若误判, 重写会固化错误。本结果是该读取裕量得到验证时的近似 ACIM 服务预算, 不是无错数字存储保证, 也不凭空添加 ECC、重复读取或重试后仍保留本数值。非破坏性读没有逐 MAC restore; 周期 refresh 单独计入。原资料未给本端点模式循环寿命保证, 寿命另列, 不将一个假定耐久系数乘入瞬时 τ 。

6 其他 gain-cell 组织为何不能混入本行

GC-01 (65 nm 2T1C) 采用读写分离、互补权重、两 bit 输入、RBL 夹位及 charge-sharing MAV, 随后由两 bit Schmitt-trigger ADC 转换。其 p.1 的完整操作流和 p.2 Fig.2-4 表明 bit 展开、夹位和 ADC 都是本地服务的一部分; 不能把低位单次 MAV 当成完整 INT8, 也不能把其读预算直接移给本例 current-programmed 3T1C。

GC-02 (28 nm mixed- V_T 4T) 测得 128×32 存储宏在 800 MHz 工作, 周期约 1.25 ns。pp.8-9 明确 5 μ s 刷新对应约 99% bit 可靠及约 93% 宏可用率; Fig.13 的 100% 样本 bit yield 曲线显著低于 99% 曲线。该 5 μ s 不能充当精确 INT8 存储的全 bit 保持保证, 更不能移植 93% 给容量大 32 倍或外围不同的设计。其 WWL 升压、感测与保持分布是器件级交叉依据, 本例既不借用其 1.25 ns 来写多级电流, 也不借用其刷新率。

GC-03 的 28 nm GEMTOO 模型显式把 256×32 行列数、读延迟与 refresh blocking 联系起来 (p.2 Fig.2); 频率取决于刷新前的残余读电流。约 10 s 的“保持饱和点”来自所设 99.9999% 可用率门限与器件设计目标, 不是已实测 oxide CIM 芯片, 也不是本硅 3T1C 的保持期。

GC-04 的八级电流状态确可作 MLC。若保留全等级, 相邻级读回再四舍五入需要约半级加噪声余量; “小于 1 ADC LSB 漂移”不能证明恢复原 code。可另设原码 shadow 并计入保存、读出及重写, 但本文不借用一个未计资源的数字 shadow。只取端点是本例的简化选择, 不是多级 gain-cell 无法工作。原 0.4 ms、>90% CIFAR10 准确率属于其四 bit 网络/映射, 也不等于本例 INT8 算法准确率。

GC-05 (16 nm 4T GC+7T STU) 的 p.2、pp.6-7 Fig.13-15 给出三个不同步骤: storage update 经 GBL/SRU 写 GC; stationary update 从 GC 读入 STU; self-refresh 由 SRU 读 RBL 再驱动 WBL。计算使用 STU, 更新 GC storage 尚未使新权重可计算。STU 与 GC 脱耦后确可边计算边刷新/更新其他 storage; 正确预算应对 GC/SRU/写口统计刷新占用, 对 STU/计算口统计提交阻塞, 允许已说明资源上的真实重叠, 不能一律相加或一律免费。其 p.9 的 1.9 ns INT8 latency 是原 16 nm 计算条件, 并非 storage 写入或 stationary 提交的完整时间; 本文未与 GC-04 的 65 ns 或 0.4 ms 拼成一条“最快”服务。

交付与复现。输入、原值/选择/推导及来源哈希保存在 data/inputs.json; 操作计数、完整分段、平均服务及资源对照在 data/results.json。脚本直接导入共享计算 API, 读取共同情景与逻辑常数, 默认只读核对; --emit 才刷新派生表。优先待电路验证的是 1/64 ns 脉冲与读噪声、128 对同时编程供流、周期内端点符号裕量以及高刷新占用调度; 这些条件不会被一个已生成的 PDF 或通过算术检查所替代。

来源识别 (短题名)。

ID	首作者/年	短题名	DOI
GC-01	S. Xie / 2022	2T1C Gain-Cell CIM	10.1109/VLSITechnologyandCircuits.2022.9830338
GC-02	R. Gitterman / 2018	800-MHz Mixed- V_T 4T IFGC	10.1109/JSSC.2018.2820145
GC-03	S. Liu / 2024	Oxide Gain Cell Design Guidelines	10.1109/TED.2024.3372938
GC-04	J. Song / 2024	Current-Programmed 3T1C MLC CIM	10.1109/JSSC.2023.3339887
GC-05	P.-C. Wu / 2025	INT-FP Dual-Mode Gain-Cell CIM	10.1109/JSSC.2024.3470215